Redes Neurais e Deep Learning





Prof. Rodrigo Caropreso
Jun/2019
rcaropreso at gmail

Esta Foto de Autor Desconhecido está licenciado em CC BY-SA

Conteúdo Programático – Aulas 1 e 2

Introdução e Redes Multi-Camadas

- Conceitos Gerais em Aprendizado Profundo
- Perceptron Multi-Camadas;
- Backpropagation e Validação Cruzada.

Redes Neurais Convolucionais

- Redes Neurais com Retardos Temporais (TDNN);
- TensorFlow: Conceitos e Exercícios
- Keras: Conceitos e Exercícios
- Redes Convolucionais: Conceitos e Exercícios

Conteúdo Programático – Aula 3

Redes Neurais Recorrentes

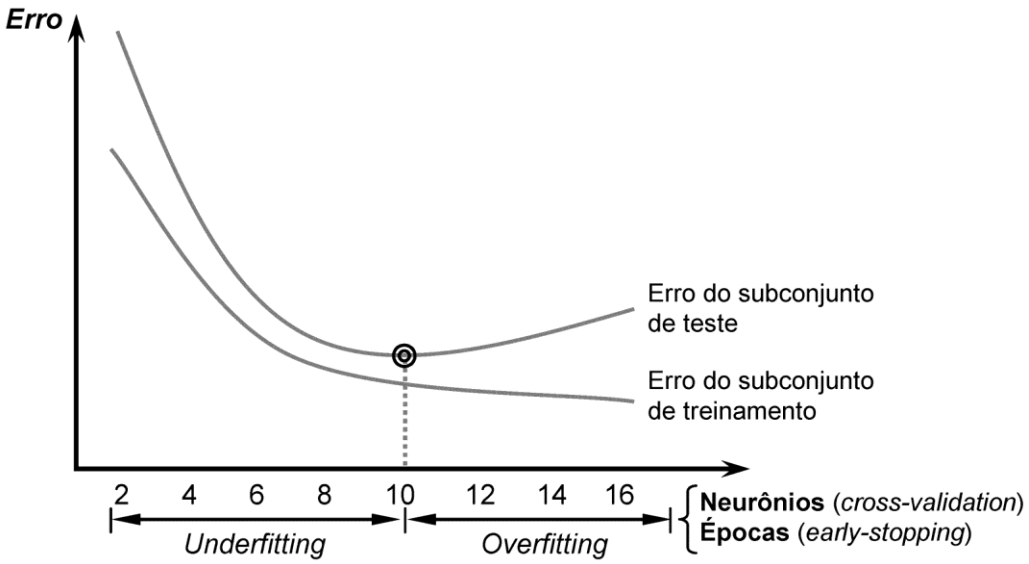
- Visão Computacional: Visão Geral
- Pytorch – Conceitos e Exercícios
- Redes de Jordan
- Redes de Elman
- Redes LSTM: Conceitos e Exercícios
- Autoencoder: Conceitos e Exercícios



CNNs: Aspectos de Treinamento

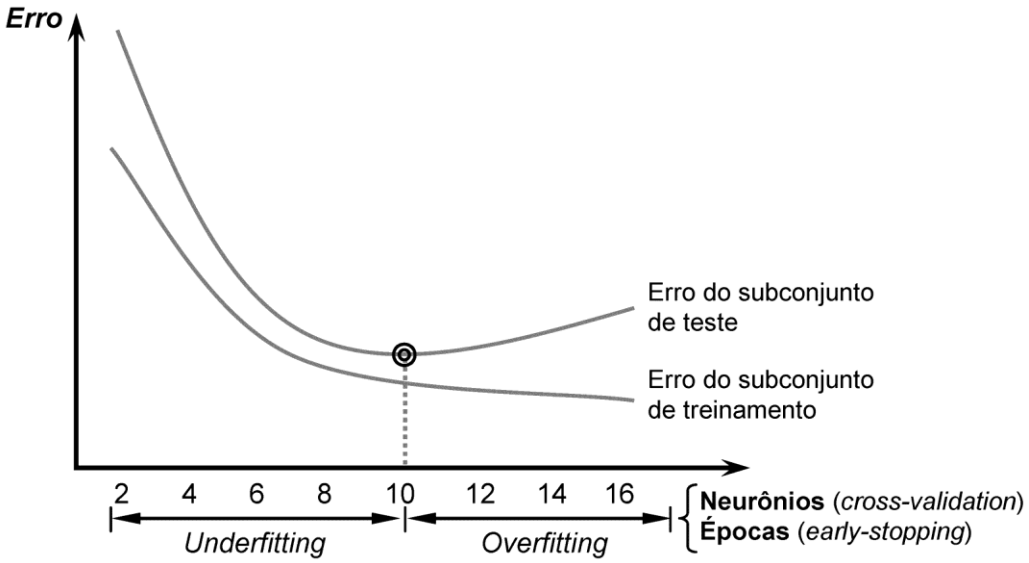
Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Aspectos de Treinamento: Regularização



- Em geral, os valores de pesos W de um modelo de aprendizado de máquina assumem valores pequenos, com bons resultados de predição;
- Quando os valores são muito elevados, podemos estar diante de um caso de overfitting!
- Neste caso recomenda-se o uso de parâmetros de Regularização (Regularization).

Aspectos de Treinamento: Regularização



Após o ajuste de $R(W)$, o algoritmo tenta minimizar $L_{reg}(W)$

- A Regularização do Modelo consiste em modificar a Função de Perda $Loss(W)$

$$L_{reg}(W) = L(W) + \lambda \cdot R(W) \rightarrow \min_{W}$$

- $L(W)$ → função de perda (MSE, Log-Loss, etc)
- $R(W)$ → regularizador (penaliza pesos elevados)
- λ → peso do Regularizador no cálculo.

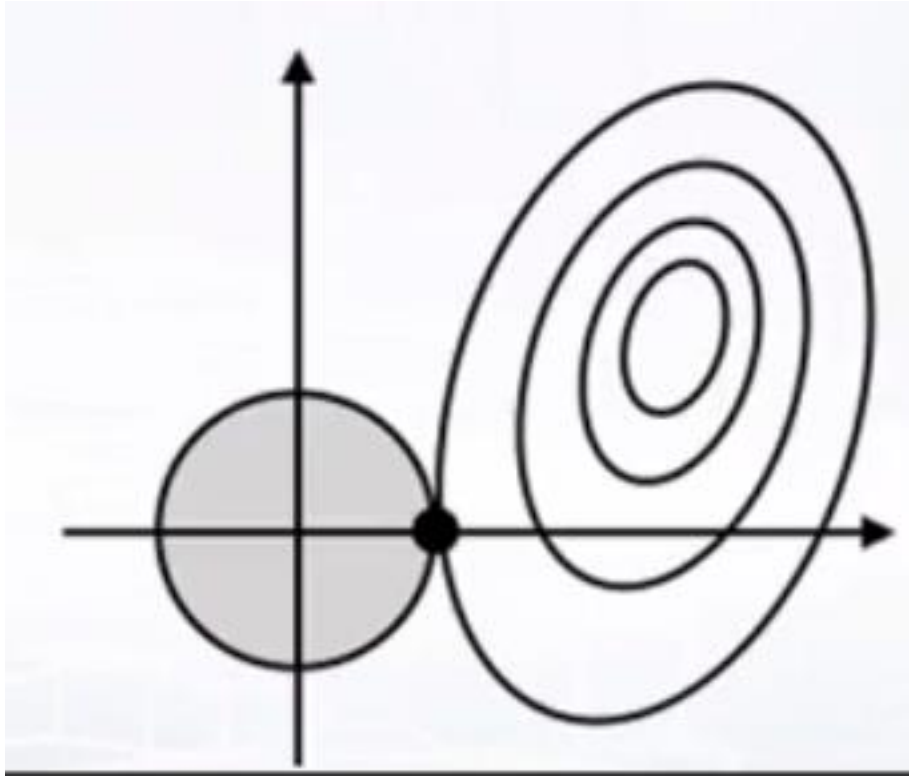
Por exemplo:

$$R(W) = \|w\|^2 = \sum_{j=1}^{d} w_j^2$$

Bias não incluído!

Aspectos de Treinamento: Regularização – L2

$$\begin{cases} L(W) \rightarrow \min\limits_{W} \\ s.t. \|w\|^2 \leq C \end{cases}$$



- Neste caso:

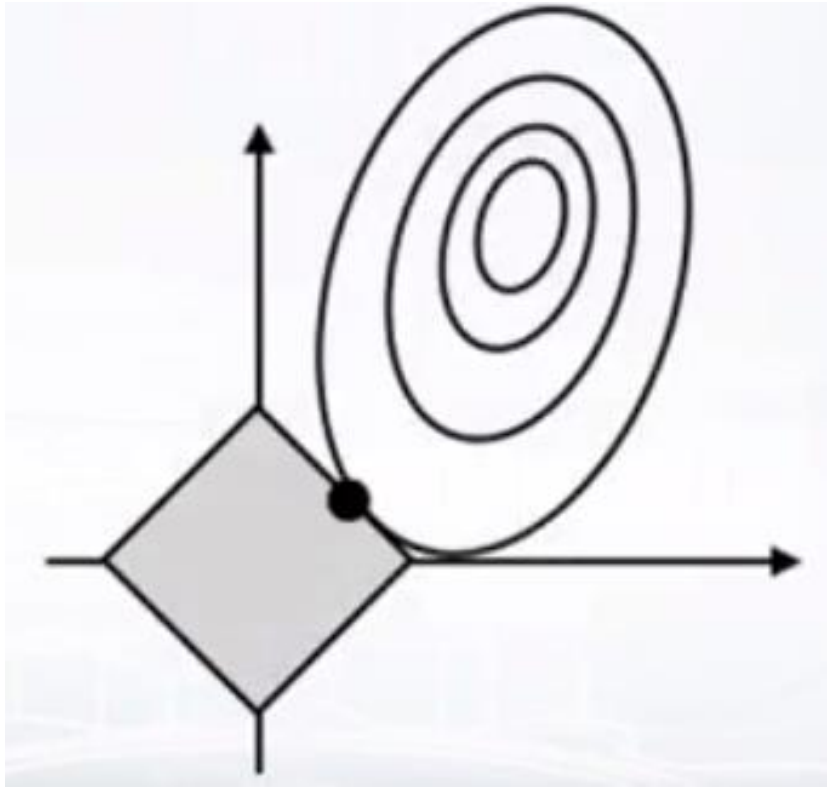
$$R(W) = \|w\|^2 = \sum_{j=1}^{d} w_j^2$$

Bias não incluído!

- A regularização L2 deixa os pesos próximos a zero
- Pode ser aplicada em algoritmos de gradiente descendente!
- Ela torna o problema matematicamente equivalente a uma otimização com restrições (constrained optimization)

Aspectos de Treinamento: Regularização – L1

$$\begin{cases} L(W) \rightarrow \min\limits_{W} \\ s.t. \|w\|_1 \leq C \end{cases}$$



- Neste caso:

$$R(W) = \|w\|_1 = \sum_{j=1}^{d} |w_j|$$

Bias não incluído!

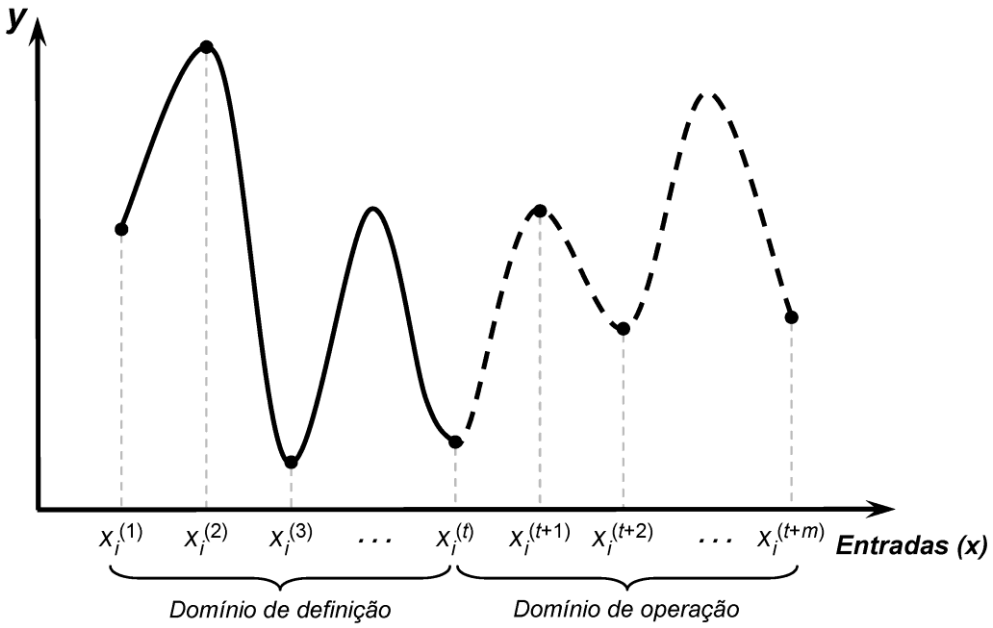
- A regularização L1 deixa alguns pesos exatamente iguais a zero;
- Consegue aprender padrões esparsos;
- Não pode ser otimizada com métodos simples de gradiente descendente.

Aspectos de Treinamento: Regularização – outras técnicas



- Redução de Dimensionalidade (PCA)
- Data Augmentation (Imagens rotacionadas, com flip, distorção, etc);
- Dropout;
- Early Stopping;
- Obter mais dados.

Stochastic Gradient Descent

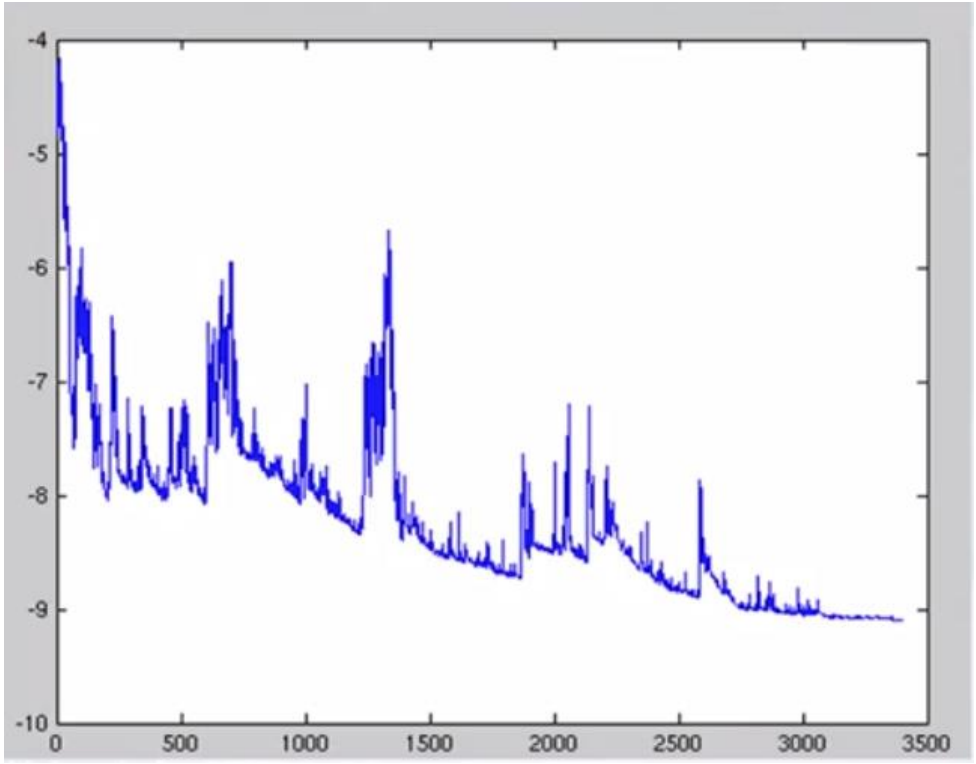


- A técnica do Gradiente Descendente realiza a otimização de uma função de Perda (Loss Function) correspondente ao custo acumulado ao longo de diversos exemplos (amostras) do conjunto de treinamento;

$$\nabla L(w) = \frac{1}{L} \cdot \sum_{i=1}^{L} \nabla (w^T x_i - y_i)^2$$

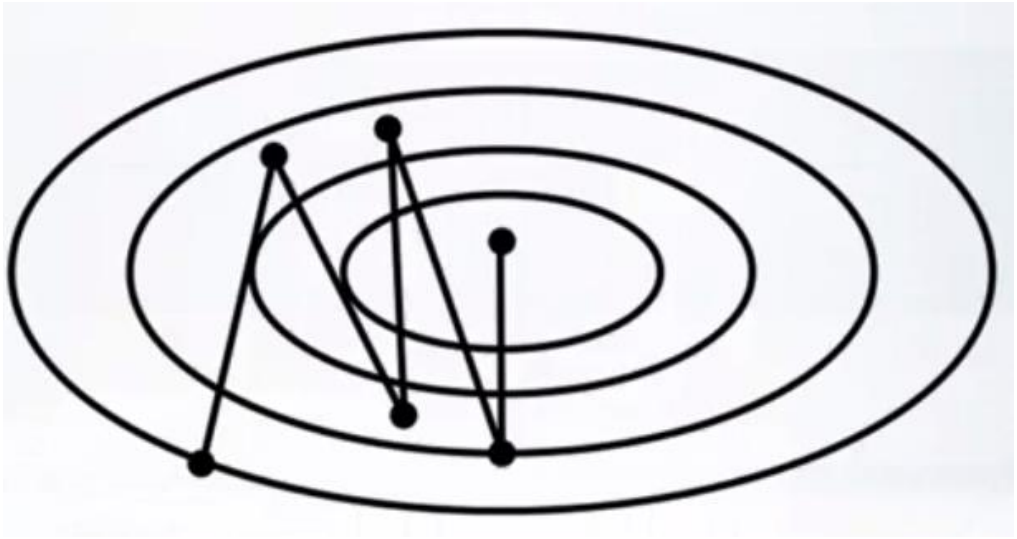
- Isso pode gerar dois problemas potenciais:
  1. Ônus computacional para problemas de grande porte
  2. Desaparecimento do Gradiente em Deep Learning

Stochastic Gradient Descent



- A técnica do Gradiente Descendente Estocástico, escolhe a cada época, para um determinado instante t, uma amostra aleatoriamente e calcula a correção do gradiente para esta amostra.
- A descida do gradiente se torna bastante ruidosa desta forma.
- Pode ser usado para treinamento online e a taxa de aprendizado deve ser escolhida com cautela.

Mini-batch Gradient Descent



- Esta técnica utiliza um pouco de cada cenário previamente estudado, ao escolher aleatoriamente um conjunto m de amostras a cada época calculando a correção do gradiente para este conjunto de m amostras.
- A correção dos pesos é realizada a partir da média do gradiente para as m amostras selecionadas.
- Desta forma a variância (ruído) da descida do gradiente se torna menor. Entretanto, a taxa de aprendizado deve ser escolhida cuidadosamente.

Adaptive Gradient



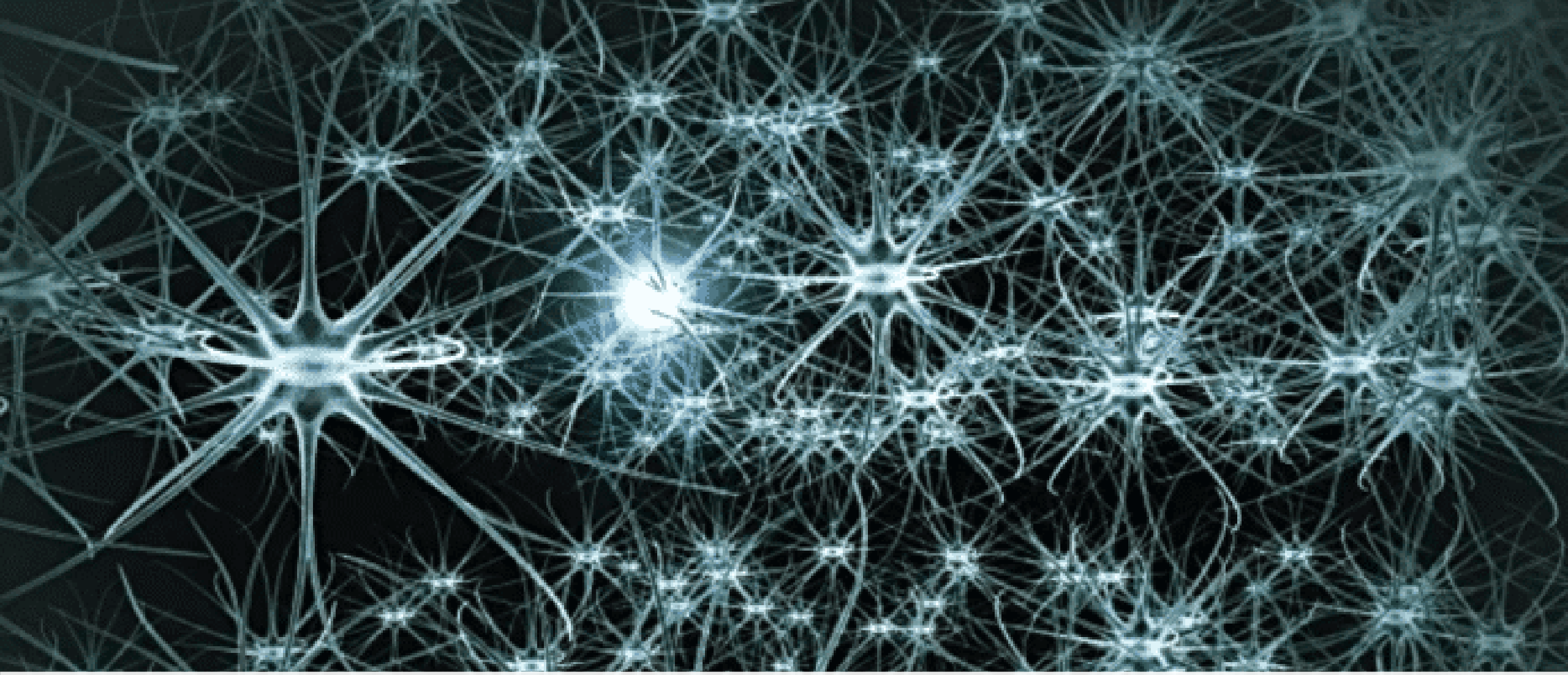
- O valor da taxa de aprendizado é ajustado ao londo das épocas:

$$\nabla_j^t = \nabla_j^{t-1} + g_{tj}^2$$

onde g_{tj} é o gradiente do j-ésimo parâmetro

$$W^t = W^{t-1} + \frac{\eta_t}{\sqrt{\nabla_j^t + \epsilon}} g_{tj}$$

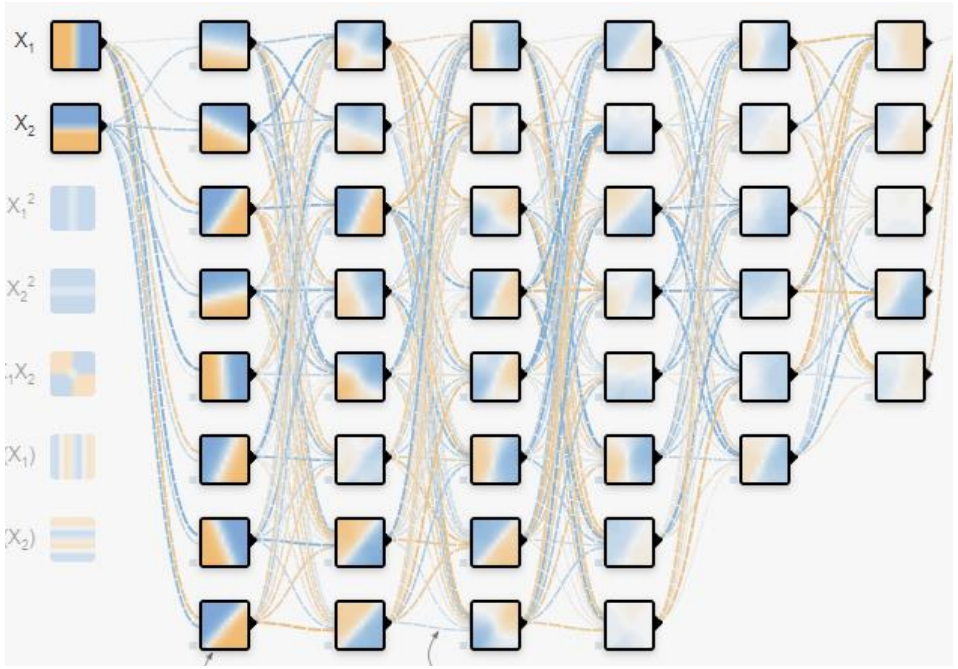
onde ϵ é um parâmetro pequeno para evitar divisão por zero.



CNN’s: Redes Neurais Convolucionais

Esta Foto de Autor Desconhecido está licenciado em CC BY-SA

CNN: Convolução - Detecção de Bordas



- São redes usadas principalmente em aplicações de Visão Computacional:
  - Veículos autônomos;
  - Detecção de pessoas;
  - Detecção de objetos
- Possui diversas camadas que realizam distintas operações nas imagens a fim de extrair características relevantes (features) que possam ser usadas em Aprendizado de Máquina.

Fonte: https://playground.tensorflow.org

CNN: Convolução - Detecção de Bordas

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

- Imagem 6x6x1 em escala de cinza;
- Para realização da operação constrói-se, por exemplo, um “filtro” (ou kernel) com dimensão 3x3:

1	**0**	**-1**
1	**0**	**-1**
1	**0**	**-1**

CNN: Convolução - Detecção de Bordas

$$3 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 0 \cdot 0 + 5 \cdot 0 + 7 \cdot 0 + 1 \cdot (-1) + 8 \cdot (-1) + 2 \cdot (-1) = -5$$

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

CNN: Convolução - Detecção de Bordas

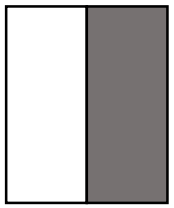
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

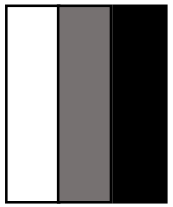
*

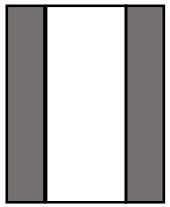
1	0	-1
1	0	-1
1	0	-1

=

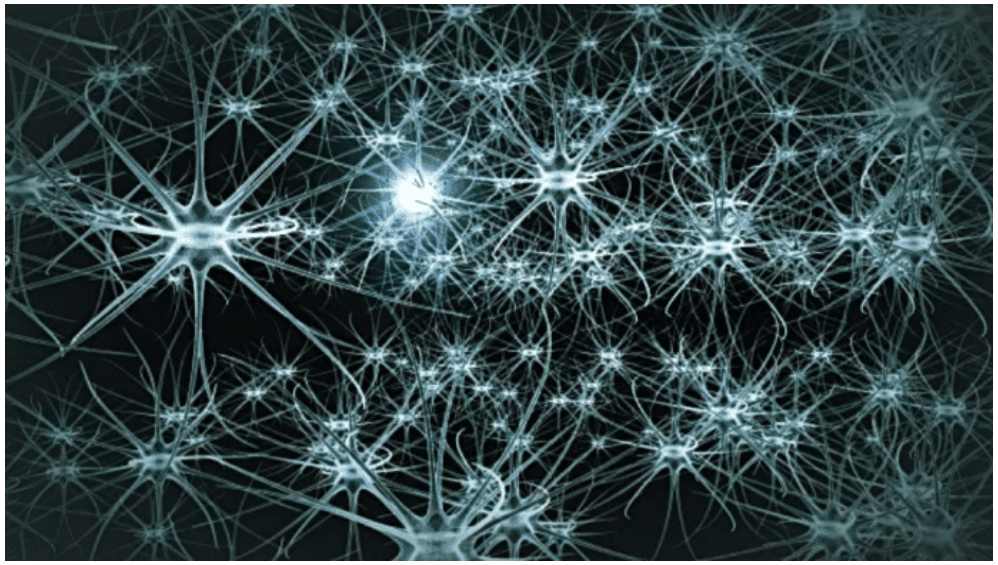
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



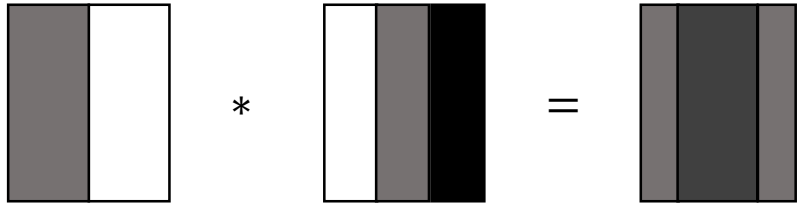




CNN: Convolução - Detecção de Bordas



- Se a sequencia de borda se inverter em relação ao slide anterior (claro → escuro para escuro → claro) então o resultado da convolução se altera:



- De forma análoga é possível definir detectores de bordas horizontais assim como tipos distintos de filtros.

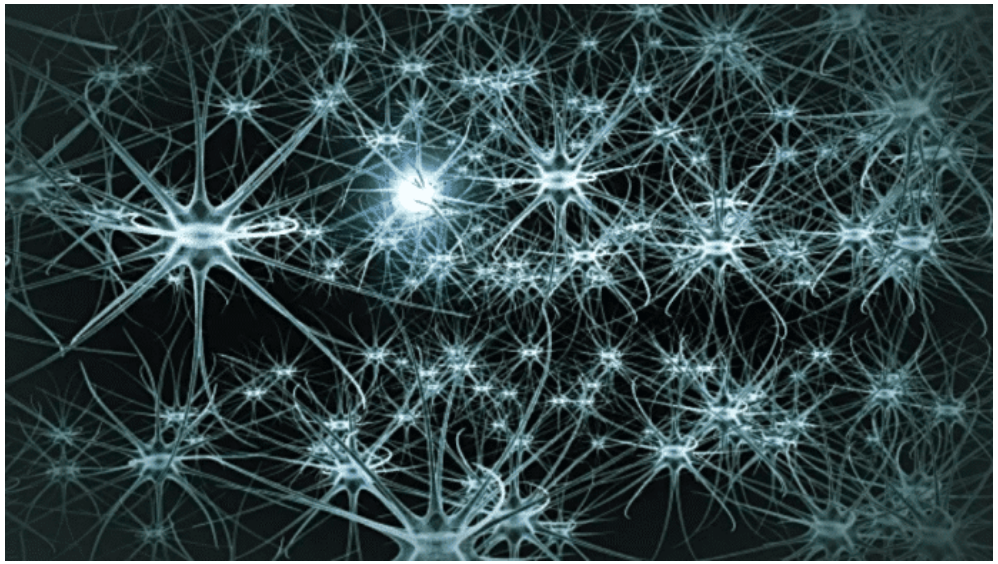
3	0	-3
10	0	-10
3	0	-3

Filtro de Scharr

1	0	-1
2	0	-2
1	0	-1

Filtro de Sobel

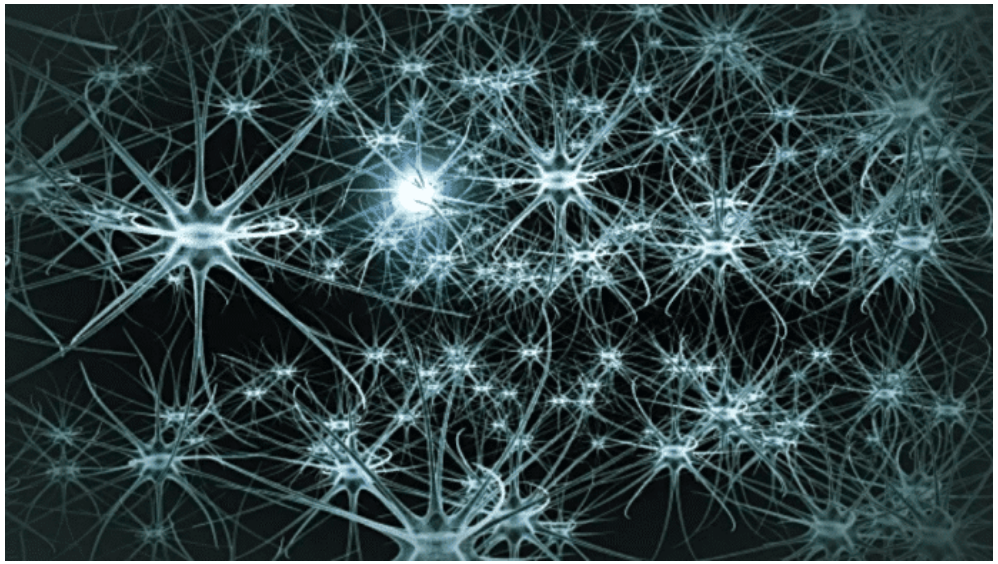
CNN: Sites



https://en.wikipedia.org/wiki/Kernel_(image_processing)

http://setosa.io/ev/image-kernels/

CNN: Convolução - Detecção de Bordas



- Em termos de Deep Learning, uma possibilidade é fazer com a rede APRENDA quais são os melhores valores dos campos do filtro para uma boa detecção de borda, ao invés de utilizar somente as opções *hardcoded*.

- Esta metodologia tem a vantagem de permitir que as redes sejam capazes de aprender características mais específicas das imagens processadas, o que nem sempre é possível com as técnicas tradicionais de Visão Computacional.

CNN: Padding

0	0	0	0	0	0
0	35	19	25	6	0
0	13	22	16	53	0
0	4	3	7	10	0
0	9	8	1	3	0
0	0	0	0	0	0

- Da convolução, lembramos que:
- Uma imagem $n\ x\ n$ convoluída com um filtro $f\ x\ f$ resulta numa imagem $\mathrm{m}\ x\ m$ onde $\mathrm{m} = n - f + 1$
- No exemplo anterior $n = 6$, $f = 3$ e $\mathrm{m} = 4$
- O Processo de **Padding** consiste em acrescentar uma borda de pixels (normalmente com valor zero, ao redor da imagem a ser processada, ANTES da convolução).

Fonte: https://blog.xrds.acm.org/2016/06/convolutional-neural-networks-cnns-illustrated-explanation/

CNN: Padding

0	0	0	0	0	0
0	35	19	25	6	0
0	13	22	16	53	0
0	4	3	7	10	0
0	9	8	1	3	0
0	0	0	0	0	0

- Na imagem da tela ao lado: $p = 1$
- Assim a imagem se torna $(n + 2p)\ x\ (n + 2p)$
- O resultado da convolução com filtro $f\ x\ f$ se torna:

$$m\ x\ m\ \rightarrow (n + 2p\ - f + 1)\ x\ (n + 2p\ - f + 1)$$

- Como $n = 4$, $p = 1$ e $f = 3$, resulta: $m = 4$, mantendo o mesmo tamanho da imagem após a operação.

Fonte: https://blog.xrds.acm.org/2016/06/convolutional-neural-networks-cnns-illustrated-explanation/

CNN: *Padding* e Convolução

0	0	0	0	0	0
0	35	19	25	6	0
0	13	22	16	53	0
0	4	3	7	10	0
0	9	8	1	3	0
0	0	0	0	0	0

- **Convolução Válida (valid)**: sem padding, ou seja:

$$p = 0$$

- **Convolução conservativa (same)**: utiliza padding de modo que o tamanho da imagem de saída seja igual ao da entrada:

$$p = variável$$

ou seja:

$$n + 2p - f + 1 = n$$

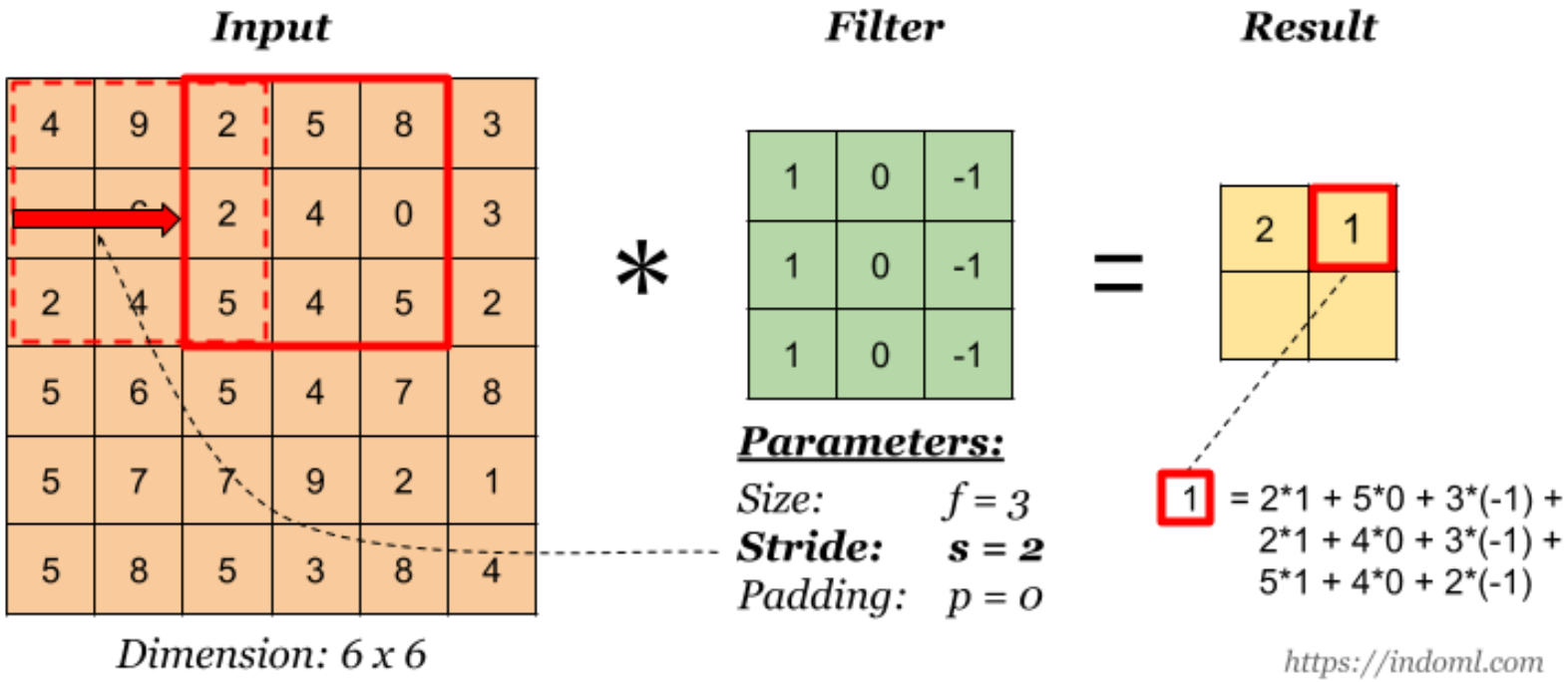
$$p = \frac{f - 1}{2}$$

f é ímpar em 99% dos casos!

Fonte: https://blog.xrds.acm.org/2016/06/convolutional-neural-networks-cnns-illustrated-explanation/

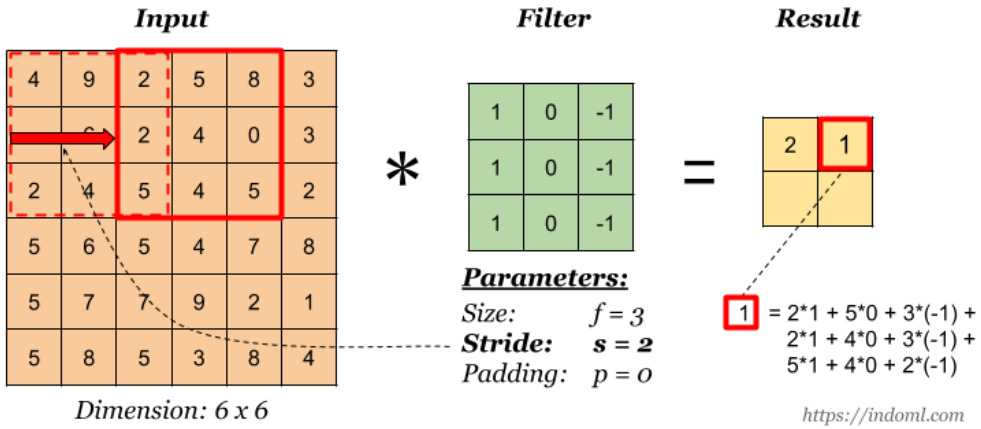
CNN: Stride e Convolução

- O passo do filtro (**Stride**) se refere ao numero de pixels que a janela de kernel do filtro deve se deslocar a cada operação.



Fonte: https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/

CNN: Fórmula Geral de Saída



- Para o cálculo do tamanho de cada imagem processada na saída de uma camada, tem-se

$$(n\ x\ n)\ *(f\ x\ f)$$

- Onde o padding é dado por p e o passo é dado por s, a imagem de saída terá o tamanho dado por:

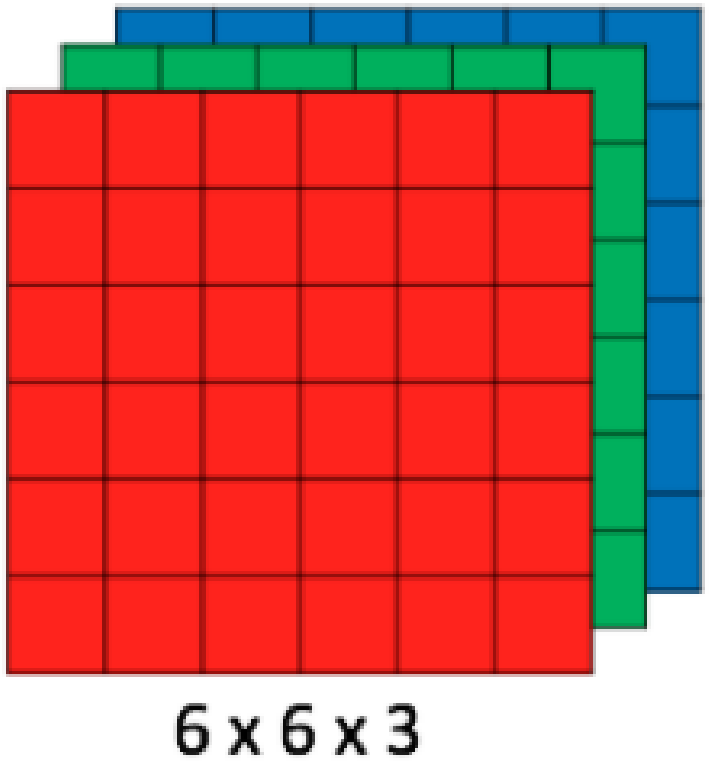
$$m = \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

- Onde o símbolo $\lfloor \ldots \rfloor$ indica que o valor deve ser arredondado para baixo (valor inteiro).

- Lembrando que a imagem será uma matriz $m\ x\ m$

Fonte: https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/2

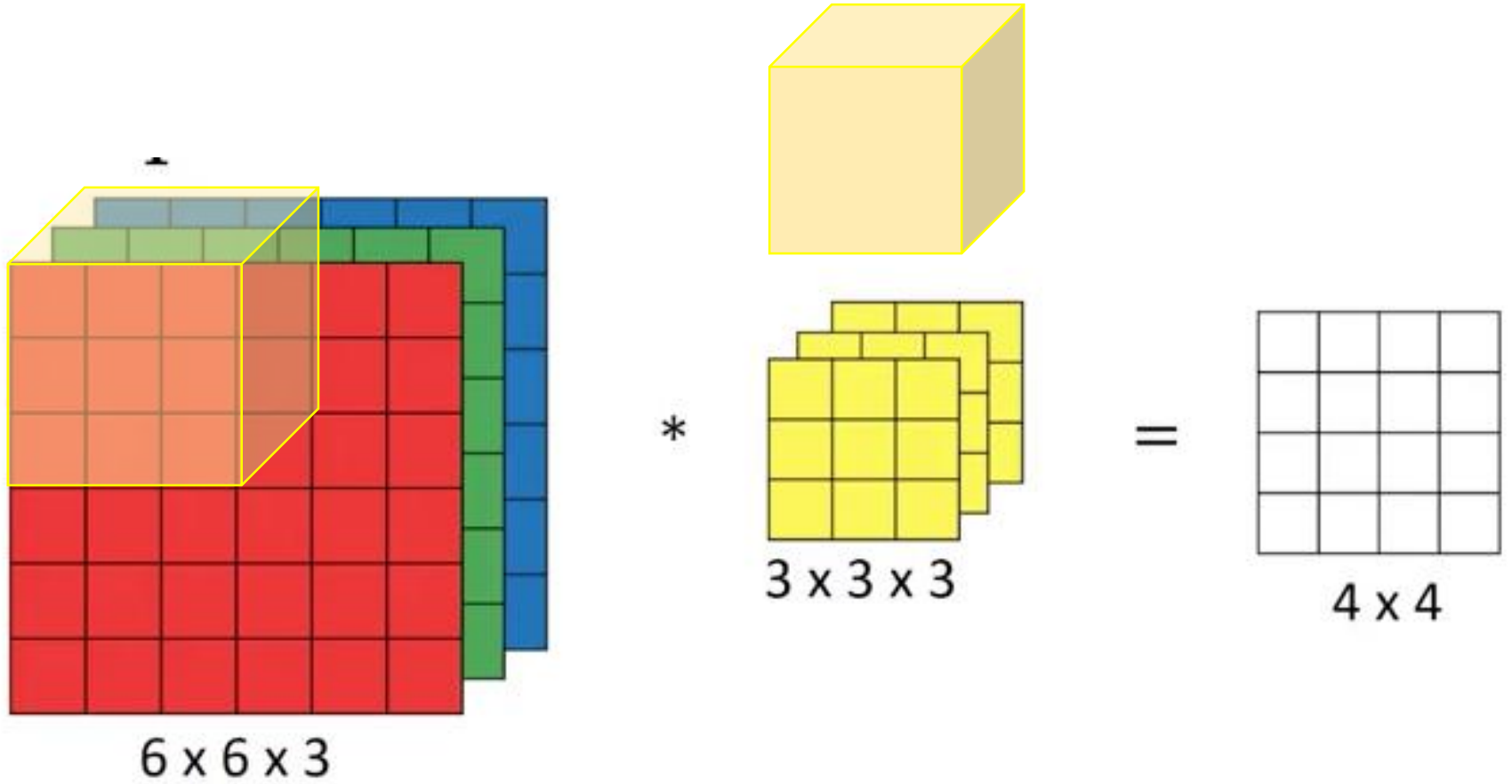
CNN: Convolução em volume



- Neste caso a imagem normalmente possui 3 canais (R,G,B) sendo a resolução representada por um Tensor do tipo $n\ x\ n\ x\ 3$.
- Neste caso, o filtro do kernel deve ter essa mesma composição, ou seja, $f\ x\ f\ x\ 3$
- Ao contrário do senso comum, a imagem de saída será $m\ x\ m$ (sem a dimensão dos canais)

Fonte: https://medium.com/@RaghavPrabhu/understanding-of-convolutional-neural-network-cnn-deep-learning-99760835f148

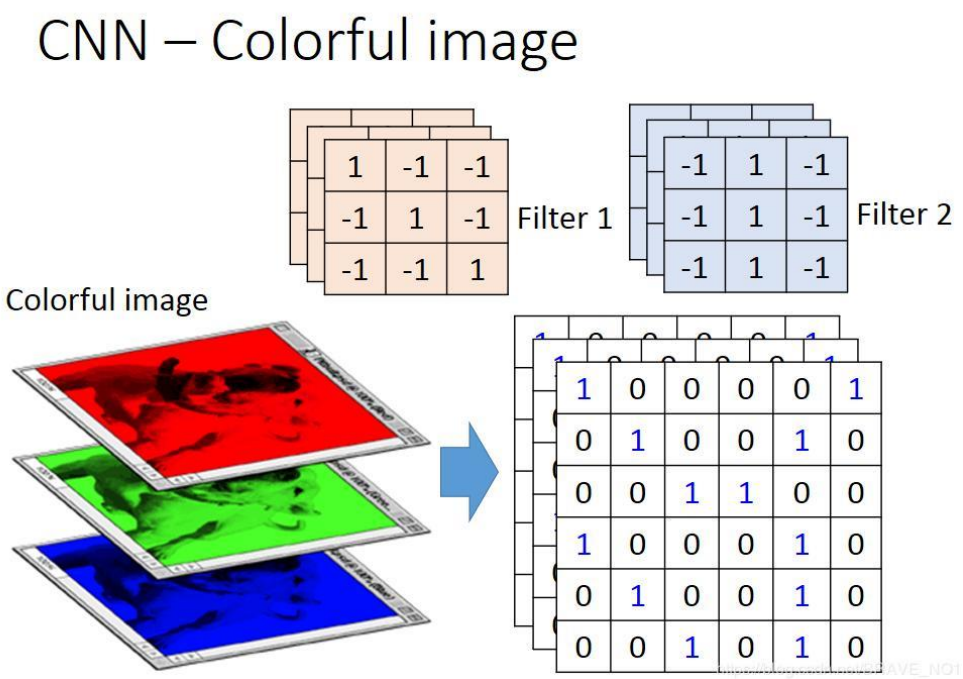
CNN: Convolução em volume



- Nesta configuração, os filtros podem ser montados para detectar features em um único canal (deixando os outros nulos) ou pode detectar features em todos os canais de cores ao mesmo tempo, dependendo de cada caso.

Fonte: https://stackoverflow.com/questions/47616700/why-in-convolutional-neural-network-3-channels-when-con-vol-into-3-channels-filt/47626225#47626225

CNN: Convolução com múltiplos filtros



Os termos 'canais' ou 'profundidade' são usados para essa dimensão $N_{filtros}$

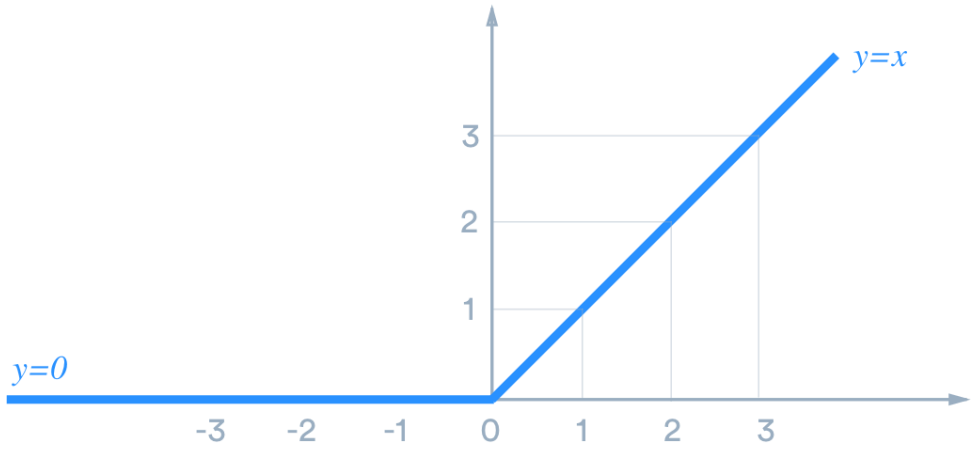
- Neste caso a imagem de saída terá uma 3ª dimensão que corresponde à quantidade de filtros usados.
- Este conjunto de imagens de saída é denominado **Mapa de Características (*Feature Map*)**.
- Desta forma:

$$(n\ x\ n\ x\ n_C) * (f\ x\ f\ x\ n_C) \rightarrow (n - f + 1\ x\ n - f + 1\ x\ n_C')$$

$$(8\ x\ 8\ x\ 3) * (3\ x\ 3\ x\ 3) \rightarrow (6\ x\ 6\ x\ N_{filtros})$$

Fonte: http://www.programmersought.com/article/8238112623/

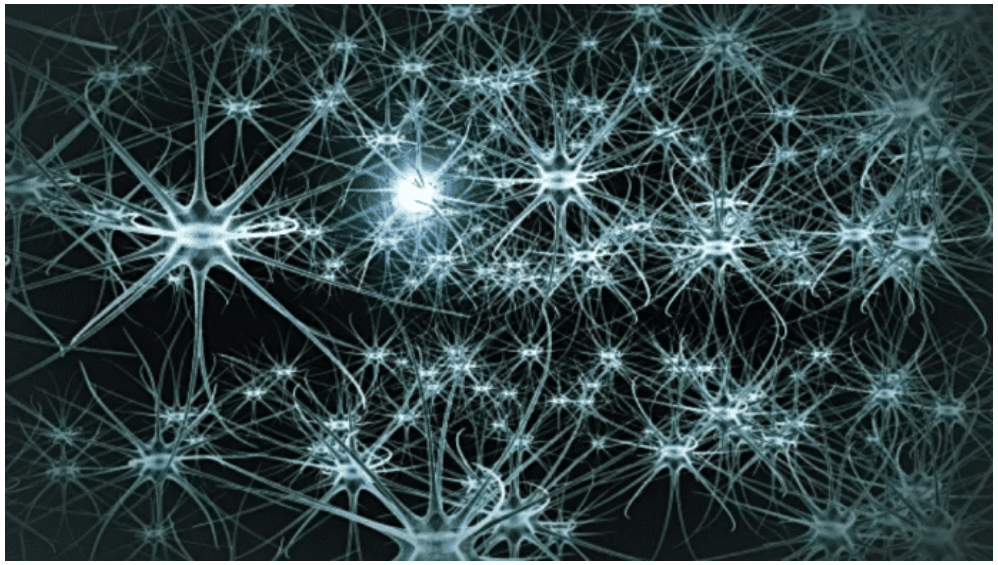
CNN: Topologia para treinamento



- Lembrando que tipicamente as camadas de uma RNA possuem a estrutura:
  1. Cálculo dos valores de entrada
  2. Adição do Bias
  3. Aplicação da Função de Ativação (não-linear)
- No contexto das CNN's, o Mapa de Características corresponde ao item 1 da lista acima.
  - Para o item 2, são adicionados $b_1, b_2, \ldots, b_{N_{filtros}}$ biases.
  - A função de Ativação mais usada é a ReLu (Rectifier Linear Unit)

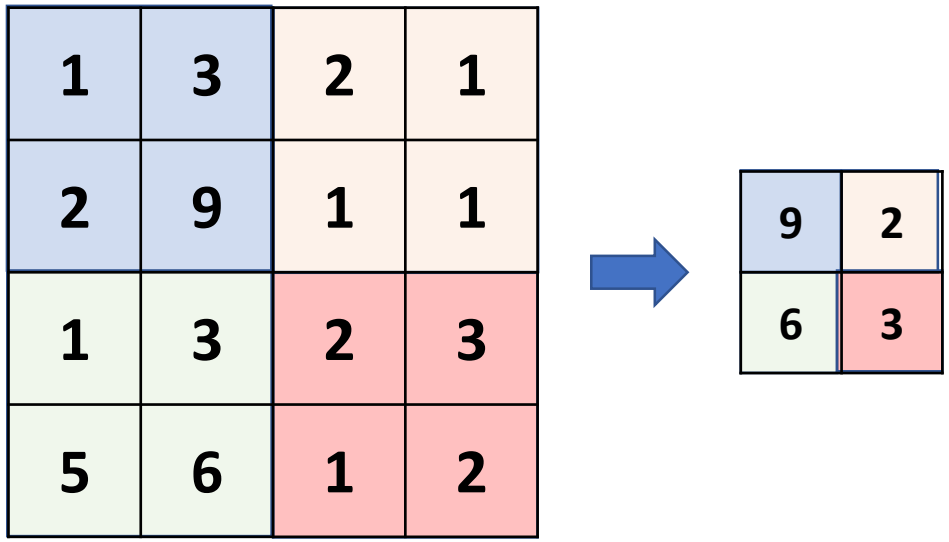
Fonte: https://medium.com/tinymind/a-practical-guide-to-relu-b83ca804f1f7

CNN: Camadas Típicas



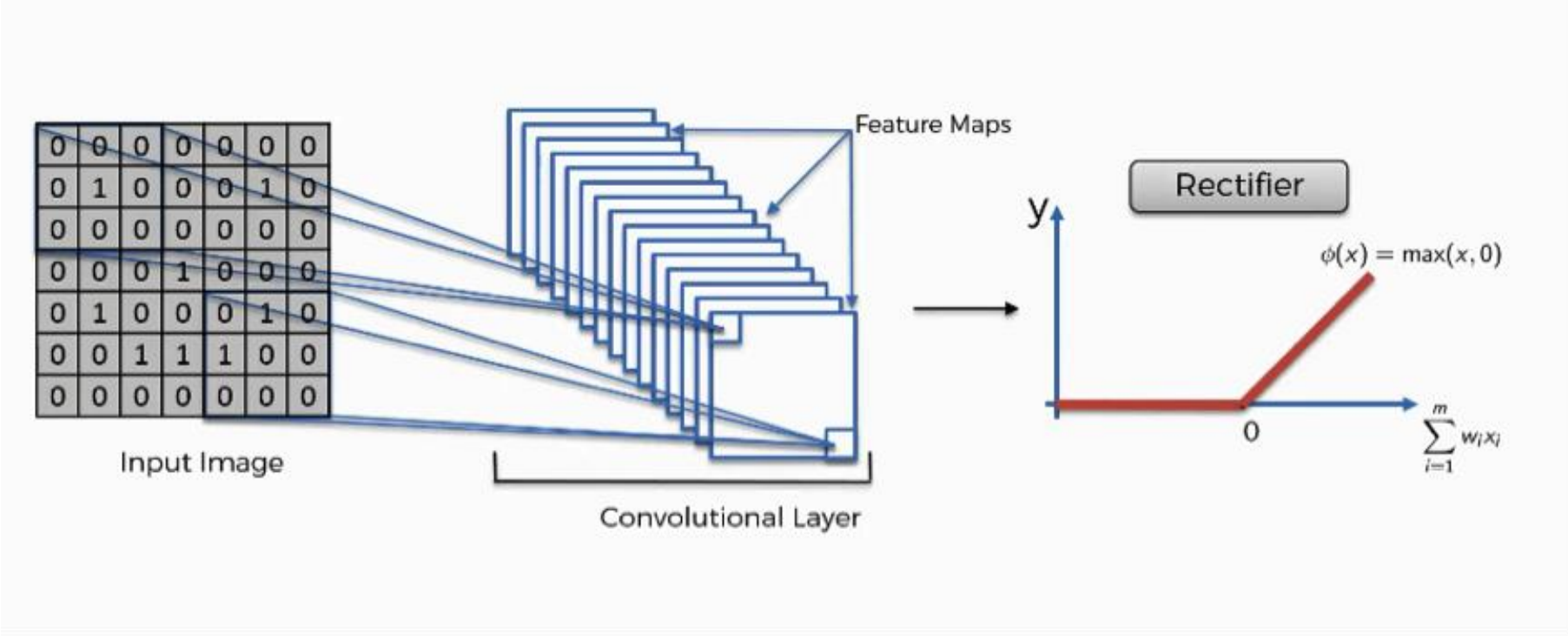
- Camada convolucional
- Camada de Pooling: acelera alguns cálculos, reduz a representação, simplificando o processamento. Ela não tem ajuste de parâmetros, seu processamento é fixo.
- O processamento em volume do Pooling (3D) é realizado independentemente para cada canal.
- Camada Densa (Fully Connected)

CNN: Max Pooling



- Hiperparâmetros:
  - $f = 2$
  - $s = 2$
- No caso do Max Pooling, normalmente não há problemas quando a janela não encaixa exatamente na imagem.
- Normalmente procura manter as principais features de ativação da imagem, mas sua aplicação primária é focada na redução de dimensionalidade.

CNN: Topologia de uma camada



Saída da Camada

- Exemplo de quantidade de parâmetros: 10 filtros de 3 x 3 x 3 na camada de entrada
- Neste caso, cada filtro terá 27 parâmetros + 1 bias = 28 x 10 filtros = 280 parâmetros na camada

Fonte: https://www.superdatascience.com/blogs/convolutional-neural-networks-cnn-step-1b-relu-layer/

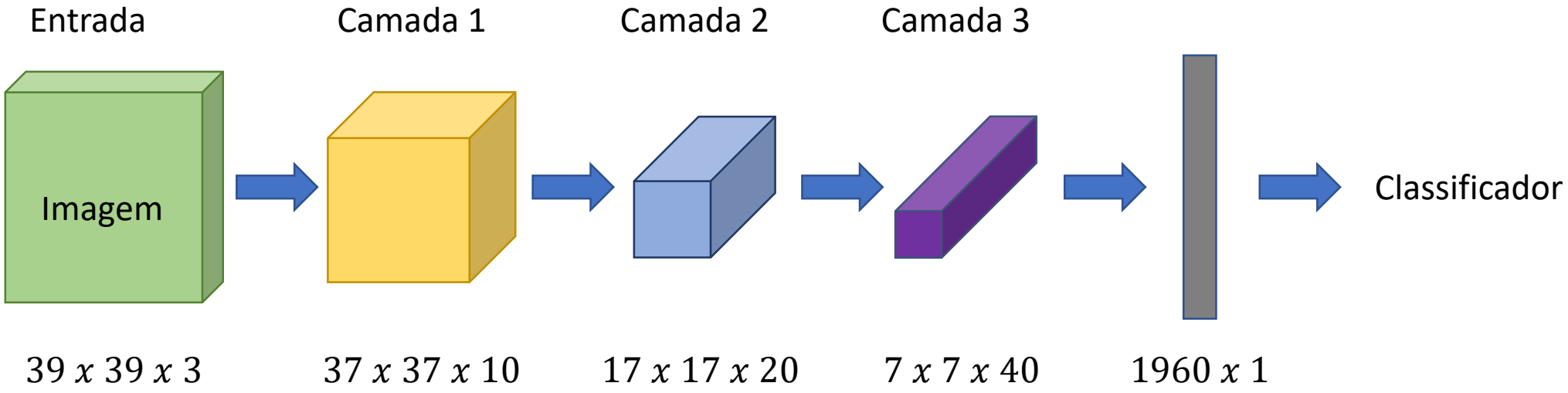
CNN: Notação para treinamento

- Dada uma camada L:
  - $f^{(L)} = tamanho\ do\ filtro$
  - $p^{(L)} = tamanho\ do\ padding$
  - $s^{(L)} = tamanho\ do\ passo\ (stride)$
  - $n_C^{(L)} = quantidade\ de\ filtros$
  - $n_C^{(L-1)} = quantidade\ de\ canais\ do\ filtro$
- Entrada da Camada L:
  - $I^{(L)} = n_H^{(L-1)}\ x\ n_W^{(L-1)}\ x\ n_C^{(L-1)}$
- Saída da Camada L:
  - $Y^{(L)} = n_H^{(L)}\ x\ n_W^{(L)}\ x\ n_C^{(L)}$
  - $n_H^{(L)} = \left\lfloor \frac{n_H^{(L-1)} + 2p^{(L)} - f^{(L)}}{s^{(L)}} + 1 \right\rfloor$

- Filtro da Camada L
  - $Dimensão = f^{(L)}\ x\ f^{(L)}\ x\ n_C^{(L-1)}$
- Pesos da Camada L:
  - $Dimensão = f^{(L)}\ x\ f^{(L)}\ x\ n_C^{(L-1)}\ x\ n_C^{(L)}$
- Bias da Camada L:
  - $n_C^{(L)}, ou\ um\ tensor\ \left[1, 1, 1, n_C^{(L)}\right]$

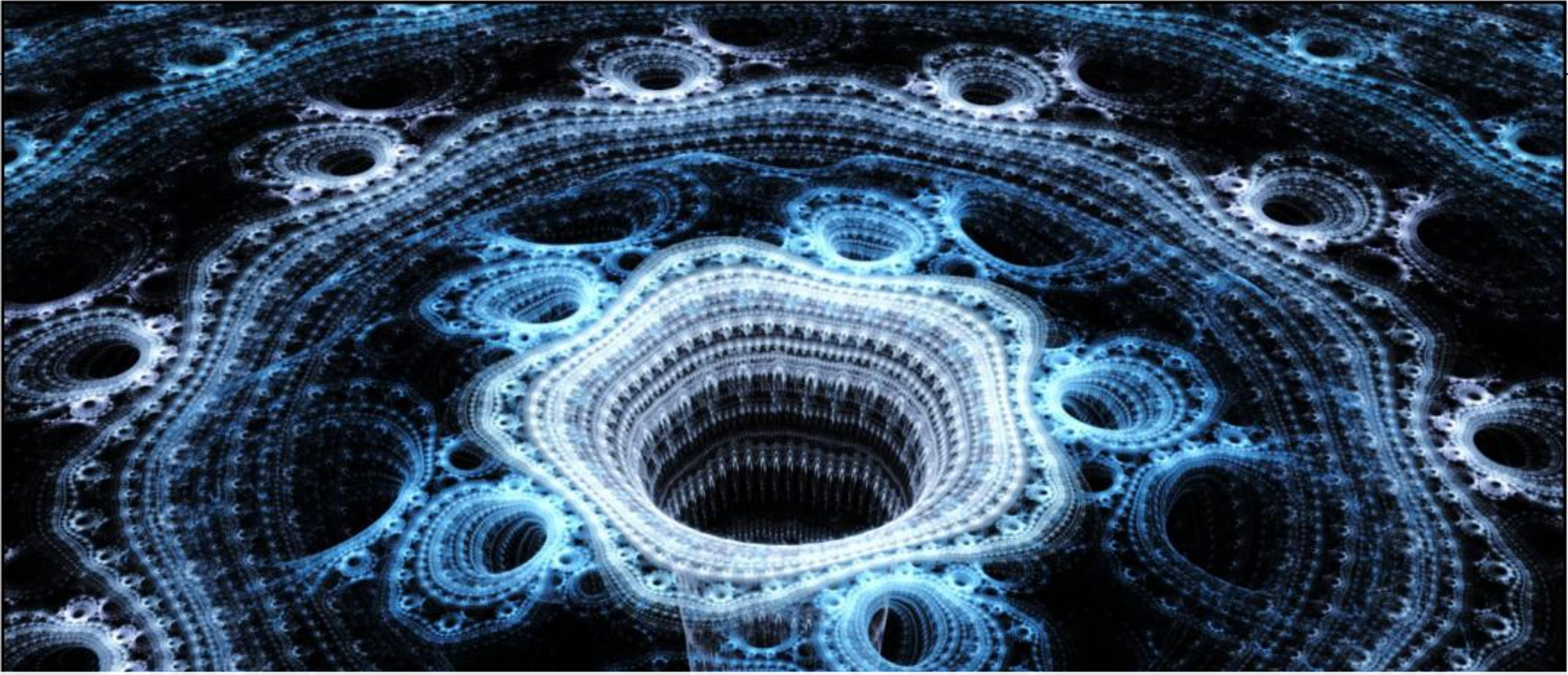
Fonte: https://medium.com/tinymind/a-practical-guide-to-relu-b83ca804f1f7

CNN: Exemplo de ConvNet



- Camada 1: zero padding, Stride = 1, Banco de 10 filtros 3x3.
  - Saída: $\frac{n+2p-f}{s}+1=\frac{39+0-3}{1}+1=37$.
  - Logo a saída será $37\ x\ 37\ x\ 10$
- Camada 2: zero padding, Stride=2, Banco de 20 filtros 5x5
  - Saída: $\frac{n+2p-f}{s}+1=\frac{37+0-5}{2}+1=17$.
  - Logo a saída será $17\ x\ 17\ x\ 20$

- Camada 3: zero padding, Stride = 2, Banco de 40 filtros 5x5.
  - Saída: $\frac{n+2p-f}{s}+1=\frac{17+0-5}{2}+1=7$.
  - Logo a saída será $7\ x\ 7\ x\ 40$

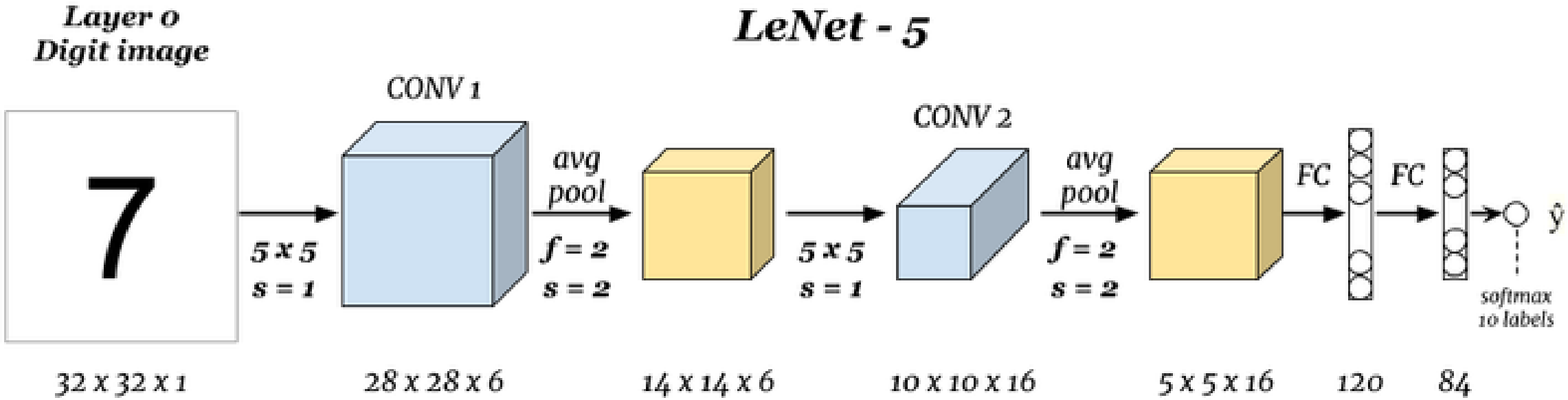


CNNs: Arquiteturas Clássicas

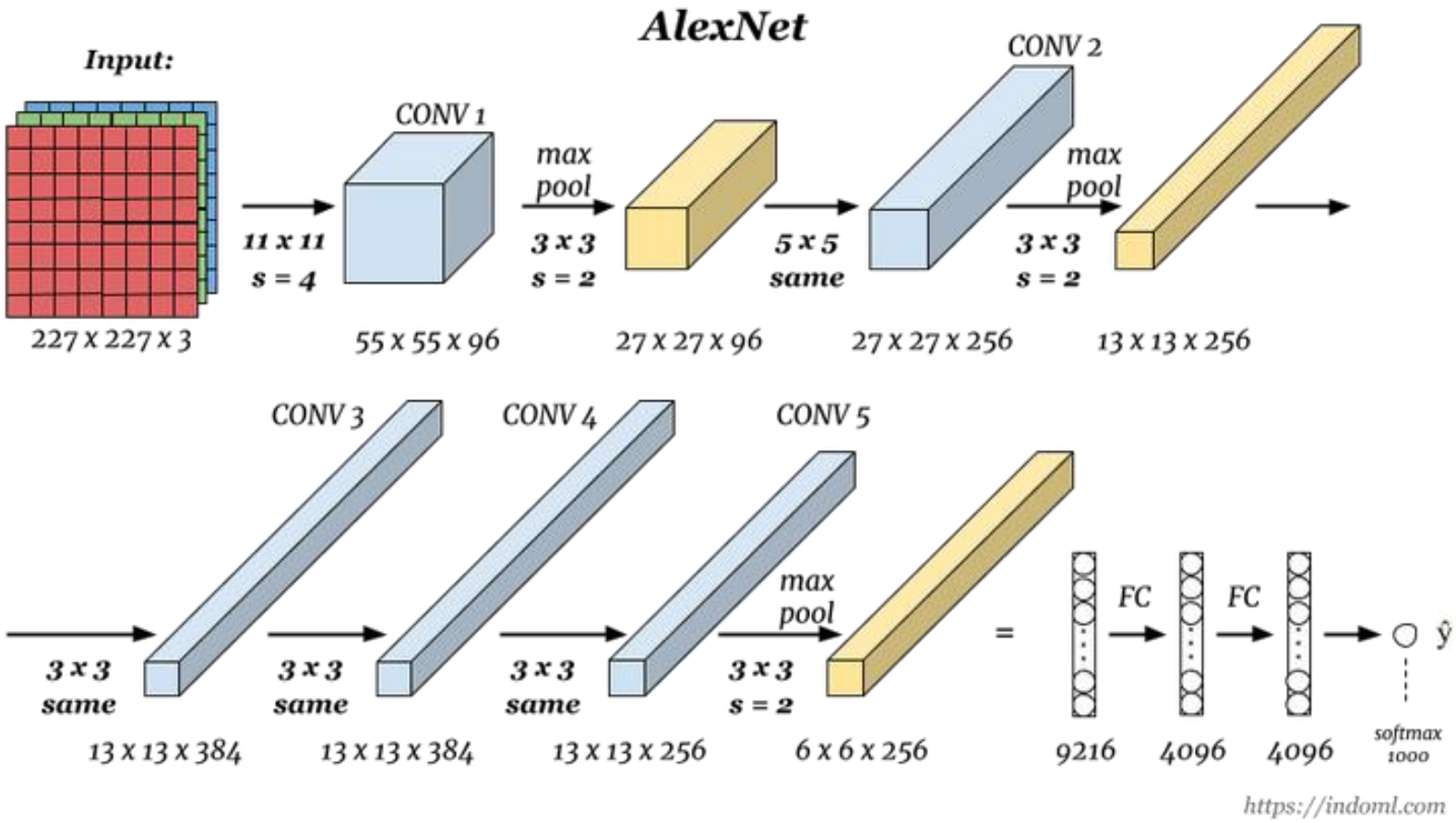
Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

CNN: LeNet - 5

- Apresentada no paper *Gradient-Based Learning Applied to Document Recognition* , de Y. Lecun, L. Bottou, Y. Bengio and P. Haffner (1998)
- Aproximadamente 60.000 parâmetros

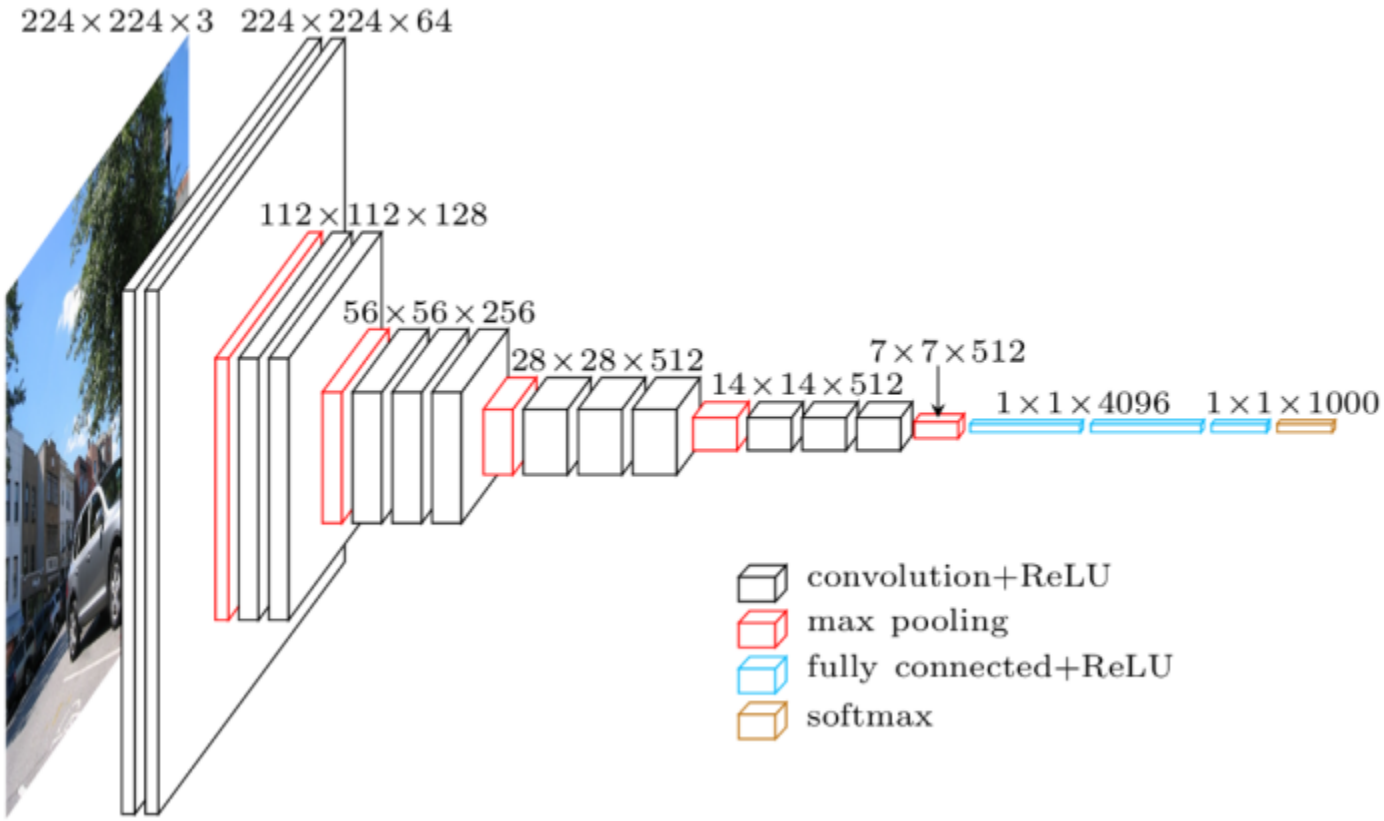


CNN: AlexNet



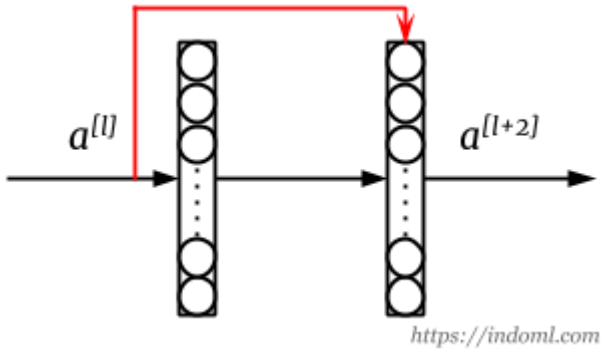
- Apresentada no paper *ImageNet Classification with Deep Convolutional Neural Networks* paper by Alex Krizhevsky, Geoffrey Hinton, and Ilya Sutskever (2012).
- Aproximadamente 60.000.000 de parâmetros

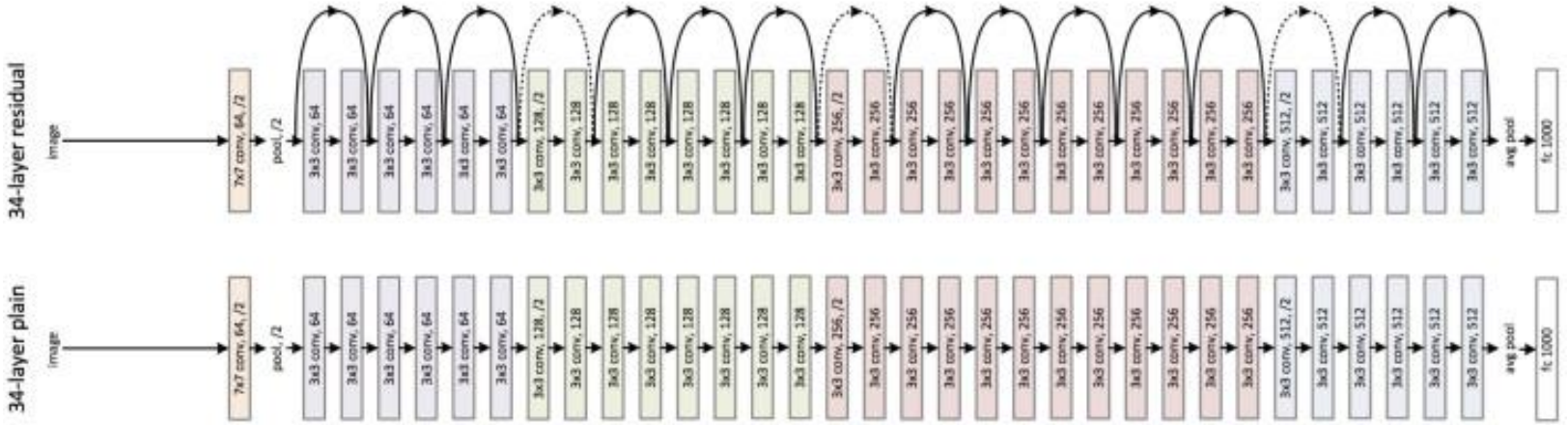
CNN: VGG-16



- Apresentada no paper *Very Deep Convolutional Networks for Large-Scale Image Recognition* paper by Karen Simonyan and Andrew Zisserman (2014)
- Aproximadamente 138 milhões de parâmetros

CNN: ResNet





- Apresentada no paper *Deep Residual Learning for Image Recognition paper* (2015)
- Utiliza uma conexão de escape que transfere a informação saltando uma camada



Tensor Flow: Conceitos e Exemplos

Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Tensor Flow



- Tensor Flow é uma biblioteca open source desenvolvida pelo Google.
- Possui backend feito em C++, por isso roda mais rápido que código Python.
- Utiliza uma estrutura denominada Data Flow Graph.
- Fornece uma API C++ e Pytthon
- Possui suporte a CPU, GPU e processamento distribuído em cluster.

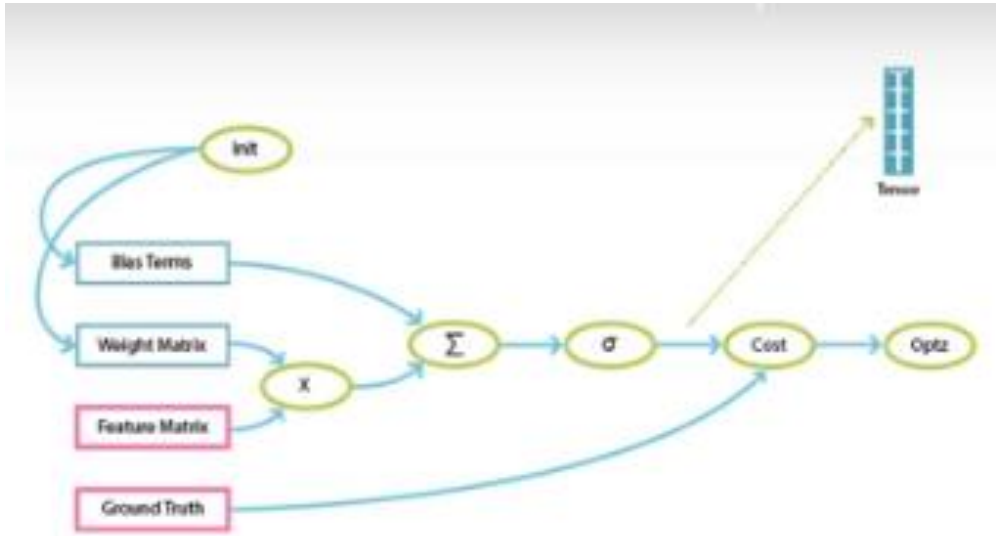
Tensor Flow



https://playground.tensorflow.org

- Data Flow Graph: possui dois elementos principais:
- Node: representa uma função matemática (operação)
- Edge: representa um array multi dimensional, conhecido como Tensor.
- Utilização padrão: envolve a construção de um grafo e sua execução após a criação de uma sessão a partir das funções ‘run’ e ‘eval’

Tensor Flow



- Após a construção do grafo, um loop interno é construido para manipular a computação
- Os dados são inseridos nos nodes do grafo a partir dos placeholders
- O Tensor Flow possui suporte nativo a deep learning e redes neurais e algoritmos de Gradiente Descendente

Tensor Flow



- É necessário abrir e fechar sessões para liberar os recursos do TF;
- É possível criar um bloco com ‘with’ e neste caso a sessão será fechada automaticamente:

```
with tf.Session() as session:
    Result = session.run(c)
    Print(result)
```

Tensor Flow



- A aparente complexidade se deve ao fato de que o TF pode fazer os cálculos de maneira distribuida, em clusters.
- O método tf.device() fornece mais informações a respeito da estrutura do tf.

Tensor Flow



- Tensor: multi-dimensional array
- (colocAR EXEMPLOS)
- São uteis para manipulação de imagens, devido às caracteristicas computacionais delas.
- Os canais de cores (RGB) são organizados de modo a compor um Tensor.

Tensor Flow: Variáveis



- tf.variable()
- Devem ser inicializadas antes que um grafo seja executado, isso pode ser feito a partir de tf.initialize_all_variables
- Atualização de valores, tf.assign()
- Para inserir dados de outros modelos, é necessário usar os placeholders

Tensor Flow: Variáveis



- Placeholders não contém valores ao serem criados, neste caso é necessário atribuir um valor depois
- É possível usar o comando tf.feed_dict para configurar os valores dos placeholders



Keras: Conceitos e Exemplos

Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Keras: Conceitos



https://keras.io/

- é uma biblioteca open source escrita em Python.
- fornece uma API de alto nível capaz de rodar os algoritmos de alguns frameworks conhecidos:
    1. TensorFlow
    2. CNTK (Microsoft Cognitive Toolkit)
    3. Theano
- possui rápida prototipagem, suportando CNN's e RNN's
- roda tanto em CPU quanto GPU.

Keras: Instalação

```
pip install tensorflow-1.7.0-cp35-cp35m-in_amd64.whl
pip install Keras-2.1.5-py2.py3-none-any.whl
```

Keras: Importação

```
import keras as K
import tensorflow as tf


py_ver = sys.version
k_ver = K.__version__
tf_ver = tf.__version__


print("Using Python version " + str(py_ver))
print("Using Keras version " + str(k_ver))
print("Using TensorFlow version " + str(tf_ver))
```

Keras: Construção do modelo



- As redes podem ser construídas de duas formas:

1. Método **Sequential()**: neste caso, a camada de entrada é implícita e o modelo começa na 1ª camada oculta.

```
model = K.models.Sequential()
model.add(K.layers.Dense(units=5, input_dim=4,
kernel_initializer=init, activation='relu'))
```

2. Classe **Model()**: neste caso, as camadas são criadas separadamente e adicionadas ao modelo

Keras: Construção do modelo



- As redes podem ser construídas de duas formas:

2. Classe **Model()**: neste caso, as camadas são criadas separadamente e adicionadas ao modelo

```
init = K.initializers.glorot_uniform(seed=1)
X = K.layers.Input(shape=(4,))
net = K.layers.Dense(units=5, kernel_initializer=init,
activation='relu')(X)
net = K.layers.Dense(units=6, kernel_initializer=init,
activation='relu')(net)
net = K.layers.Dense(units=3,
kernel_initializer=init,activation='softmax')(net)
model = K.models.Model(X, net)
```

Keras: Compilação do modelo



- Após a declaração do modelo, ele deve ser compilado antes da realização do treinamento:

```
model.compile(loss='categorical_crossentropy',
optimizer=simple_adam, metrics=['accuracy'])
```

- Definir o parâmetro ‘loss’ os tipos de otimizadores
- ***metrics***: opcional, contendo uma lista de parâmetros indicando índices de desempenho que devem ser calculados durante a fase de treino.

Keras: Treinamento do Modelo



- Após a compilação do modelo, ele pode ser treinado:

```
b_size = 1
max_epochs = 10
print("Starting training ")
h = model.fit(train_x, train_y, batch_size=b_size,
epochs=max_epochs,
shuffle=True, verbose=1)
```

- ***batch_size***:
- ***shuffle***: determina se as amostras serão escolhidas aleatoriamente durante o treinamento
- O método fit(.) retorna um dicionário contendo o histórico do treinamento

Keras: Treinamento do Modelo



- O método fit(.) possui um parâmetro callback, que pode ser usado para direcionar os eventos do treinamento para uma classe externa processar.

```
h = model.fit(train_x, train_y,
batch_size=batch_size, epochs=max_epochs,
verbose=0, callbacks=[my_logger])

class MyLogger(K.callbacks.Callback):
    def on_epoch_end(self, epoch, logs={}):
        if epoch % self.n == 0:
        curr_loss = logs.get('loss')
        total_acc = my_accuracy(self.model,
        self.data_x,
        self.data_y, self.pct_close)
        ...
```

Keras: Verificação do Modelo



- o histórico do treinamento pode ser consultado para fornecer detalhes sobre o treinamento

```
print(h.history['loss'])
```

- Os resultados podem ser avaliados pelo método evaluate(.):

```
eval = model.evaluate(test_x, test_y, verbose=0)
print("Avaliação dos dados de teste: loss = %0.6f
accuracy = %0.2f%% \n" \% (eval[0], eval[1]*100) )
```

- eval[0] contém o valor da função de perda passada para a função compile; os demais valores da lista tem relação direta com o parâmetro ***metrics*** passado em compile(.)

Keras: Operação do Modelo



- A partir do método predict(.) é possível realizar a operação do modelo:

```
np.set_printoptions(precision=4)
unknown = np.array([[6.1, 3.1, 5.1, 1.1]],
dtype=np.float32)
predicted = model.predict(unknown)
print(“Usando o modelo para previsão de especie: ")
print(unknown)
print("\nA espécie é da classe: ")
print(predicted)
```

Keras: Aspectos de Treinamento - Classificação



- Classificação Multiclasse:
  1. Fazer o ajuste das classes de saída (**one-hot-encoding**)
  2. Função de ativação da camada de saída do tipo **softmax** (podendo ser interpretado como probabilidade).
  3. Função de perda do tipo **categorical_crossentropy** (é possível usar **mean_squared_error**)
  4. O parâmetro **accuracy** pode ser passado como métrica à função **compile(.)**
  5. Otimizador do tipo **Adam** pode ser usado

Keras: Inicializadores

Function	Description
`Zeros()`	All `np.float32 0.0` values
`Constant(value=0)`	All a single specified `np.float32` value
`RandomNormal(mean=0.0, stddev=0.05, seed=None)`	Gaussian, bell-shaped distribution
`RandomUniform(minval=-0.05, maxval=0.05, seed=None)`	Random, evenly distributed between `minval` and `maxval`

- Inicializadores são úteis para gerar os valores iniciais das matrizes de pesos e bias de Redes Neurais construídas como modelo.

Keras: Inicializadores

Function	Description
`glorot_normal(seed=None)`	Truncated Normal with `stddev = sqrt(2 / (fan_in + fan_out))`
`glorot_uniform(seed=None)`	Uniform random with limits `sqrt(6 / (fan_in + fan_out))`

- Inicializadores devem ser passados para as camadas durante a construção do modelo

```
init = K.initializers.glorot_uniform(seed=1)
```

Keras: Funções de Ativação

Function	Descripton
`relu(x, alpha=0.0, max_value=None)`	if x < 0 , f(x) = 0, else f(x) = x
`tanh(x)`	hyperbolic tangent
`sigmoid(x)`	f(x) = 1.0 / (1.0 + exp(-x))
`linear(x)`	f(x) = x
`softmax(x, axis=-1)`	coerces vector **x** values to sum to 1.0 so they can be loosely interpreted as probabilities

- Funções de ativação que podem ser utilizadas para a camada densa (fully Connected) de uma RNA

```
model.add(K.layers.Dense(units=5, input_dim=4, kernel_initializer=init, activation='relu'))

model.add(K.layers.Activation('relu'))
```

Keras: Funções de Perda (Loss Functions)

Function	Description
`binary_accuracy(y_true, y_pred)`	For binary classification
`categorical_accuracy(y_true, y_pred)`	For multiclass classification
`sparse_categorical_accuracy(y_true, y_pred)`	Rarely used (see documentation)
`top_k_categorical_accuracy(y_true, y_pred, k=5)`	Rarely used (see documentation)
`sparse_top_k_categorical_accuracy(y_true, y_pred, k=5)`	Rarely used (see documentation))

- Funções de perda que são usadas como métricas durante os treinamentos de uma RNA

```
model.compile(loss='mean_squared_error', optimizer=simple_sgd, metrics=['mse'])
```

Keras: Otimizadores

Optimizer	Description
`SGD(lr=0.01, momentum=0.0, decay=0.0, nesterov=False)`	Basic optimizer for simple neural networks
`RMSprop(lr=0.001, rho=0.9, epsilon=None, decay=0.0)`	Often used with recurrent neural networks, very similar to Adadelta
`Adagrad(lr=0.01, epsilon=None, decay=0.0)`	General purpose adaptive algorithm

- Funções de otimização que podem ser usadas durante os treinamentos de uma RNA.

```
model.compile(loss='mean_squared_error', optimizer=simple_sgd, metrics=['mse'])
```

Keras: Otimizadores

Optimizer	Description
`Adadelta(lr=1.0, rho=0.95, epsilon=None, decay=0.0)`	Advanced version of Adagrad, similar to RMSprop
`Adam(lr=0.001, beta_1=0.9, beta_2=0.999, epsilon=None, decay=0.0, amsgrad=False)`	Excellent general-purpose, adaptive algorithm

- O Adadelta é uma das variações mais avançadas do Gradiente Descendente Estocástico.

```
model.compile(loss='mean_squared_error', optimizer=simple_sgd, metrics=['mse'])
```

Keras: Dropout



- O objetivo do dropout é reduzir a possibilidade de overfitting.
- Ele indica a porcentagem de nós da camada anterior que serão desconsiderados naquela época de treinamento.
- Se torna mais um hiperparâmetro a ser avaliado.

Keras: ReLu



- A função ReLu tende a ser mais resistente ao problema de Desaparecimento de Gradiente (*Vanishing Gradient*).

- **Vanishing Gradient**: como a atualização dos pesos depende do gradiente, o valor deste vai decrescendo ao longo das camadas, de modo que, em redes muito profundas, pode ocorrer a interrupção da atualização dos pesos a partir de um certo ponto.

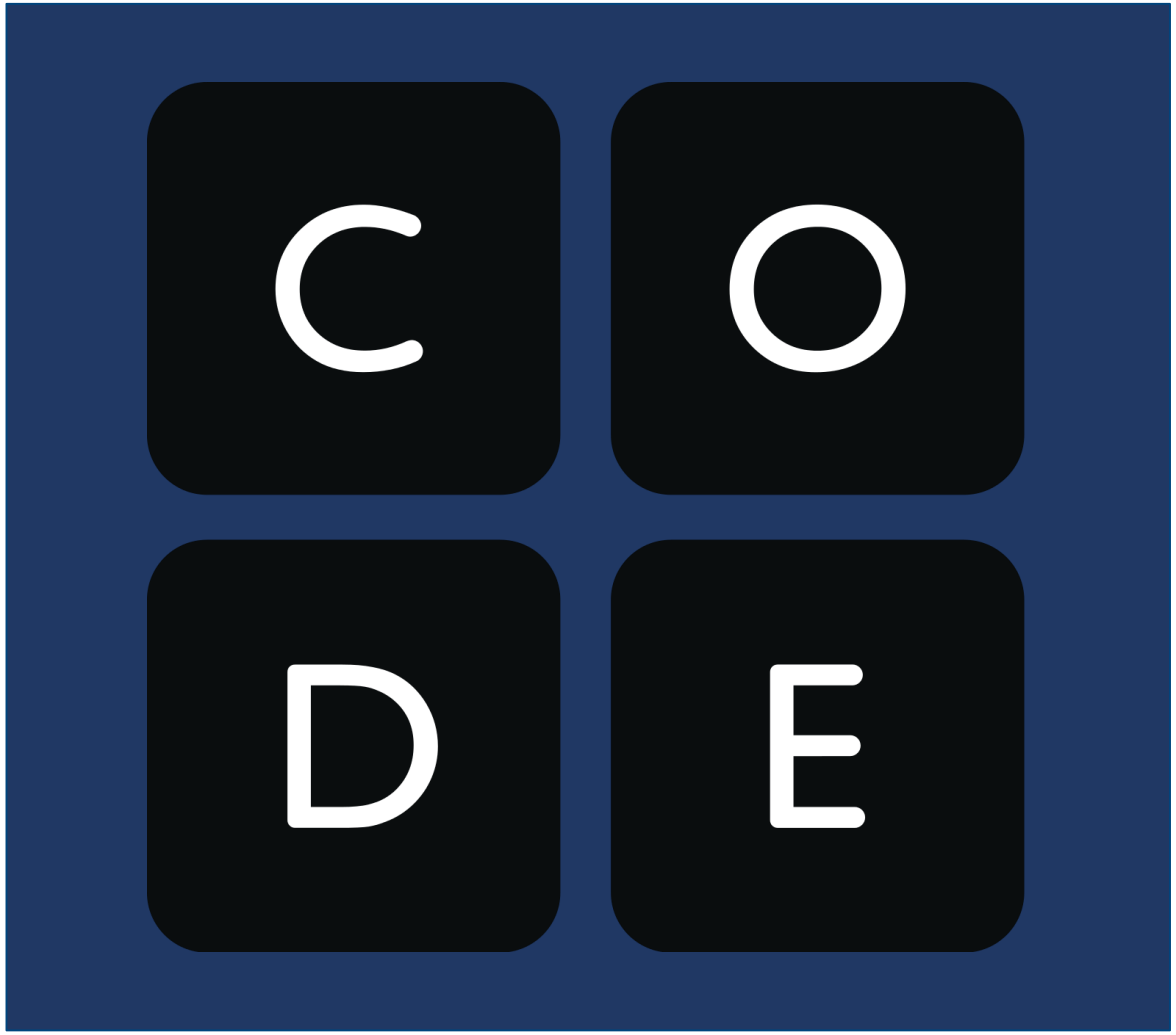
Keras: Binary_crossentropy



- A função de perda **binary_crossentropy** é mais recomendada para problemas de classificação binária e sua formulação matemática é baseada na Regressão Logística.

- Para problemas de classificação multi-classes, é recomendado o uso de **categorical_crossentropy** ou **mean_squared_error**.

Challenge Time





Referências



- RUSSELL, STUART J.. Inteligência Artificial. 2ª ed. São Paulo: Campus - Elsevier, 2004.
- FERNANDES, Anita M. R.. Inteligência Artificial: noções gerais. 1ª ed. Florianópolis: Visual Books, 2008. Capítulo 1.
- SILVA, I.N. – Sistemas Inteligentes – Notas de Aula da disciplina de Redes Neurais Artificiais – USP São Carlos, 2012.
- SILVA, I.N., SPATTI, D.H., FLAUZINO, R.A., Redes Neurais Artificiais para Engenharia e Ciências Aplicadas. 1ª ed. São Paulo: Editora Artiliber, 2010.
- https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/



Facens

AQUI TEM ENGENHARIA